

“VOICE CONTROLLED WHEEL CHAIR “

A Project report submitted in the partial fulfillment the award of degree of

BACHELOR OF TECHNOLOGY

IN

Electronics and Communication Engineering

BY

D. ANUSHA

REGNO: 231801130004

N. POORNESH

REGNO: 231801130018

N. YAMUNA

REGNO: 231801130007

K. RAMU

REGNO:231801400012

P. JOHN

REGNO: 231801410011

V.MAHENDRA

REGNO:231801440001

T.SRI VARSHITH

REGNO:231801130022

Under the esteemed Guidance of

Dr. K. Ramya
Assistant Professor



Centurion
UNIVERSITY

Shaping Lives...
Empowering Communities...

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT

ANDHRA PRADESH

TEKKALI VILLAGE, TEKKALI MANDAL 535003

(2024-2025)

CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT

ANDHRA PRADESH

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**



**Centurion
UNIVERSITY**

*Shaping Lives...
Empowering Communities...*

BONAFIDE CERTIFICATE

This is to certify that the project work entitled “**VOICE CONTROLLED WHEEL CHAIR**” of project work done by D. ANUSHA (231801130004), N. POORNESH (231801130018), N. YAMUNA (231801130007), K. RAMU (231801400012), P. JOHN(231801410011), V.MAHENDRA(231801440001),T.SRIVARSHITH(231801130022) Under the esteemed Guidance of for the award the Degree of **BACHELOR OF TECHNOLOGY in ELECTRONICS AND COMMUNICATION ENGINEERING, CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT**, during the academic year **2024-2025**.

INTERNAL GUIDE

Dr. K. Ramya
Assistant Professor
Dept. of ECE

HEAD OF THE DEPARTMENT

Dr. K. Jogi Naidu
Assistant Professor
Dept. of ECE

Dr. D. Srinivasa Rao Ph.

DEAN SoET

Associate Professor

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

It is with at most pleasure and excitement we submit our project partial fulfillment of the requirement for the award of Bachelor of Technology.

The project is a result to the cumulate efforts, support, guidance, encouragement and inspiration from many of those for whom we have to give our truthful honor and express gratitude through bringing out this project at the outset as per our knowledge.

We convey my special thanks to our project Guide **Dr. K. Ramya, Assistant Professor**, who has guided, encouraged and tremendously supported me to enhance my knowledge with present working of this project to bring out enriching the quality of project.

We express my appreciativeness to **Dr. K. Jogi Naidu**, Head of the Department of ECE, who facilitated us to providing the friendly environment which helped to enhance my skills in present project.

We thank **Dr. D. Srinivasa Rao** Ph.D, Dean of SoET, Vizianagaram Campus for their invaluable guidance, insightful feedback, and continuous support throughout the course of this project. Your expertise and mentorship have been invaluable.

We convey my sincere thanks to **Dr. P. Pallavi, Registrar of Centurion University of Technology and Management**, who provided us with an opportunity to take on project work in well-equipped laboratories of Computer Science Department in our college.

At the outset, we thank to **Prof. Prashanth Kumar Mohanty** beloved **Vice Chancellor of Centurion University of Technology and Management** who is the back bone by providing for completion of this project, Thank you sir.

DECLARATION

We hereby declare that the project entitled "**VOICE CONTROLLED WHEEL CHAIR** " submitted to the fulfilment of award the Degree of **B. TECH (ECE)** at **CENTURION UNIVERSITY OF TECHNOLOGY AND MANAGEMENT (A.P)**. This project work in original has not been submitted so far in any part or full for any other university or institute for the award of any Degree or Diploma.

D. ANUSHA	(231801130004)
N. POORNESH	(231801130018)
N. YAMUNA	(231801130007)
K. RAMU	(231801400012)
P. JOHN	(231801400011)
V. MAHENDRA	(231801440001)
T.SRIVARSHITH	(231801130022)

ABSTRACT

The proposed Voice-Controlled Wheelchair project presents an innovative, low-cost mobility solution aimed at assisting individuals with physical disabilities by integrating speech-based control, obstacle detection, and microcontroller-driven automation. Built around the STM32 microcontroller platform, the system interprets voice commands received via a Bluetooth module to enable real-time movement control, allowing users to navigate forward, backward, left, or right with minimal physical effort. Safety is enhanced through an ultrasonic sensor that continuously monitors the environment and prevents collisions by triggering automatic braking when obstacles are detected.

The hardware architecture incorporates a dual-motor driver configuration to control the wheelchair's movement, along with an emergency stop switch for immediate manual shutdown. Feedback mechanisms such as buzzer alerts ensure reliable operation, while the firmware manages smooth motor transitions and command validation. A modular design makes the system adaptable to different wheelchair frames, motor specifications, and additional assistive sensors.

On the software side, a mobile-based voice interface communicates with the STM32 through wireless transmission, ensuring seamless interpretation of user commands. The system architecture supports future expansion into app-based control, cloud logging, or integration with medical assistive networks.

This project showcases the convergence of embedded systems, assistive technology, and human-machine interaction, emphasizing affordability, safety, and user-centric design. With its accessible control method and robust obstacle-avoidance mechanism, the Voice-Controlled Wheelchair serves as an effective prototype for next-generation mobility solutions in healthcare and rehabilitation environments.

TABLE OF CONTENTS

1. INTRODUCTION	1
1.1. BACKGROUND.....	1
1.2. PROBLEM STATEMENT	2
1.3. OBJECTIVES	2
1.4. SCOPE OF THE PROJECT	3
1.5. ORGANIZATION OF THE REPORT	4
2. LITERATURE SURVEY.....	5
2.1. REVIEW OF EXISTING WORK.....	5
2.2. COMPARISON/ANALYSIS	5
2.3. RESEARCH GAP	6
3. METHODOLOGY	7
3.1. TOOLS & TECHNOLOGIES USED	7
3.2. SYSTEM DESIGN.....	7
3.3. ARCHITECTURE	8
3.4. SOFTWARE AND TOOLS:	8
3.5. WORKFLOW	10
4. IMPLEMENTATION	10
4.1. MODULES DESCRIPTION	10
4.2. ALGORITHMS/PROCESSES	10
4.3. SOURCE CODE:.....	11
5. RESULTS AND DISCUSSION.....	20
5.1. ADMIN PANEL	ERROR! BOOKMARK NOT DEFINED.
6. CONCLUSION & FUTURE SCOPE.....	21
6.1. CONCLUSION.....	21
6.2. FUTURE SCOPE.....	21
7. REFERENCE	21

INTRODUCTION

The rise of automation and intelligent embedded systems has revolutionized assistive technologies, enhancing mobility and independence for individuals with physical disabilities. Traditional wheelchairs, which depend on manual effort or joystick control, often limit usability for those with restricted motor functions. To overcome these challenges, this project presents a Voice-Controlled Wheelchair that integrates speech recognition, embedded control, and real-time automation to provide a hands-free, affordable, and reliable mobility solution. Designed with scalability in mind, it offers an accessible option for healthcare environments, rehabilitation centers, and personal use.

At the core of the system is the STM32F401RE microcontroller, which serves as the main controller for interpreting voice commands and controlling movement. The wheelchair responds to simple spoken instructions such as “Forward,” “Backward,” “Left,” “Right,” and “Stop.” A speech recognition module (like Elechouse VR3 or EasyVR) processes the user’s voice and sends corresponding digital data to the STM32 via UART communication. The microcontroller then generates PWM signals to motor driver circuits (BTS7960 or L298N) that control the DC motors. Safety is enhanced using ultrasonic sensors for obstacle detection and an emergency stop switch that halts the system instantly. Additional components, including LED indicators and a buzzer, provide feedback for successful operation.

The overall design combines efficient hardware and intelligent software control to ensure precision, responsiveness, and user safety. The wheelchair is powered by a 12V/24V battery system with regulated 5V output for sensors and the controller, ensuring stable and uninterrupted operation. Its modular structure allows easy customization for advanced features such as IoT-based health tracking, AI-powered navigation, or GPS integration. This project exemplifies how embedded systems and automation can be harnessed to create socially impactful solutions that promote independence and improve the quality of life for differently-abled individuals.

1.1. Background

Wheelchairs have long been essential mobility aids, enabling independence and accessibility for individuals with physical disabilities. Traditional wheelchairs, however, depend on manual propulsion or joystick-based control, which can be challenging for users with limited upper-body strength or motor impairments. These systems often lack automation, adaptive safety features, and integration with modern assistive technologies. As embedded systems, automation, and voice-recognition technologies continue to advance, there is a growing opportunity to redesign wheelchairs into smarter, more interactive, and user-friendly assistive devices that enhance both mobility and independence.

To align with these technological trends, the Smart Vending Machine project introduces a solution that eliminates coin mechanisms entirely by integrating UPI-based payment and IoT communication. It leverages the ESP32S3WROOM microcontroller to handle multi-channel operations, touch screen input, and network connectivity. Unlike conventional setups, this design is modular, educationally adaptable, and low-cost—ideal for academic demonstrations and small-scale implementations. Combined with features such as IR-based item detection, web-based user control, and automated stock monitoring, the system provides a modernized and scalable platform for future vending solutions that emphasize convenience, hygiene, and efficient management.

1.2. Problem Statement

Traditional wheelchairs rely primarily on manual propulsion or joystick-based control, which limits usability for individuals with severe physical disabilities or restricted motor control. These systems often demand continuous physical effort, making them impractical for patients with limited hand movement or muscle strength. Furthermore, conventional designs lack automation, safety intelligence, and adaptive control mechanisms, resulting in reduced independence and increased dependency on caregivers. The absence of modern assistive technologies such as voice recognition and sensor-based obstacle detection further restricts accessibility and convenience for users.

This project aims to design a Voice-Controlled Wheelchair that overcomes these challenges by integrating speech recognition, microcontroller-based automation, and real-time safety features. Utilizing an STM32F401RE microcontroller, the system

interprets voice commands to control the wheelchair's movement, enabling hands-free navigation in all directions. Ultrasonic sensors are employed for obstacle detection, while an emergency stop mechanism ensures user safety. Feedback components like LEDs and buzzers provide confirmation of received commands. This intelligent, low-cost approach enhances accessibility, safety, and user independence—demonstrating how embedded systems and automation can revolutionize assistive mobility technology.

1.3.Objectives

- **Voice Command-Based Control:** Enable hands-free wheelchair operation through speech recognition, allowing users to issue simple commands such as Forward, Backward, Left, Right, and Stop.
- **Microcontroller Integration:** Utilize the STM32F401RE microcontroller to process voice inputs, generate control signals, and manage real-time movement through PWM-based motor control.
- **Obstacle Detection and Safety:** Incorporate ultrasonic sensors to detect nearby obstacles and implement an emergency stop mechanism to ensure user safety during navigation.
- **Motor and Sensor Coordination:** Design reliable control logic for DC motor drivers (BTS7960/L298N) to achieve smooth, directional movement and provide immediate feedback using LEDs and buzzers.
- **System Efficiency and Scalability:** Develop a low-cost, energy-efficient, and modular system architecture that can be easily expanded with additional features such as IoT connectivity, GPS tracking, or AI-based health monitoring.

1.4.Scope of the Project

Inclusions:

- Development of a voice-controlled wheelchair system using the STM32F401RE microcontroller for real-time voice command processing and motor control.
- Integration of a speech recognition module (such as Elechouse VR3 or EasyVR) to interpret user commands for direction and movement.
- Implementation of ultrasonic sensors for obstacle detection and emergency stop mechanisms to ensure user safety.
- Control of DC motors through motor driver circuits (BTS7960/L298N) using PWM signals for precise and smooth movement.

- Use of LEDs and buzzers to provide feedback for command acknowledgment and system alerts

Exclusions:

- Integration with AI-based autonomous navigation or path planning systems.
- IoT or cloud connectivity for remote health monitoring or location tracking (future enhancement).
- Support for multi-language or adaptive speech recognition (limited to predefined English commands).
- Commercial manufacturing and large-scale deployment beyond prototype demonstration.

Limitations:

- The prototype responds only to a fixed set of five basic voice commands: Forward, Backward, Left, Right, and Stop.
- The speech recognition accuracy may reduce in noisy or echo-prone environments.
- Battery capacity limits continuous operation to short durations.
- The system currently lacks autonomous obstacle avoidance beyond basic detection and stops functionality.
- Designed primarily for educational and prototype purposes, not for certified medical or commercial use

1.5.Organization of the Report

This report presents the development of the Voice-Controlled Wheelchair project in a clear and systematic manner. It begins with an introduction that outlines the project's background, problem statement, objectives, and scope. The following sections discuss existing research on assistive mobility systems and describe the methodology, including the tools, technologies, and system design used to integrate voice recognition with the STM32 microcontroller for real-time control and automation.

The later sections explain the implementation process, covering hardware assembly, software programming, and system testing. The results and discussion highlight the wheelchair's performance, accuracy, and safety features under various operating conditions. Finally, the report concludes with key findings and recommendations for future improvements such as IoT

integration, AI-based navigation, and enhanced voice recognition to make the system more efficient and accessible.

2. LITERATURE SURVEY

2.1.Review of Existing Work

Past research and development in assistive mobility have primarily focused on improving the design, efficiency, and control of traditional wheelchairs. Early systems relied on manual propulsion or joystick-based mechanisms, which provided mobility but required significant physical effort and coordination. Later advancements introduced electric-powered wheelchairs with motorized control, offering greater ease of movement but still demanding hand-based operation. Researchers have also explored gesture-controlled and brain-computer interface (BCI) systems to provide alternative input methods, though these often involve complex circuitry and high implementation costs, limiting their accessibility for widespread use.

Recent developments have shifted toward the integration of voice recognition, embedded systems, and IoT technologies to enable hands-free control and improved safety. Several prototypes using Arduino, Raspberry Pi, and STM32 platforms have demonstrated the feasibility of using speech commands to navigate. Some studies incorporated ultrasonic sensors for obstacle detection and Bluetooth modules for remote control. However, many existing designs face challenges such as limited voice accuracy, environmental noise interference, lack of safety redundancy, and high cost. The proposed Voice-Controlled Wheelchair aims to overcome these issues by combining accurate voice command processing, real-time obstacle detection, and a low-cost embedded design to create a more practical and reliable assistive mobility solution.

2.2.Comparison/Analysis

Modern assistive mobility systems have evolved to include advanced control technologies such as joystick-based navigation, gesture recognition, brain–computer interfaces (BCI), and smart sensor integration, offering improved functionality and comfort compared to traditional manual wheelchairs. However, these advanced models often come with high development costs, complex design, and limited accessibility for users with severe physical disabilities.

The proposed Voice-Controlled Wheelchair provides a balanced and affordable solution by integrating speech recognition, microcontroller-based motor control, and safety features such as obstacle detection and emergency stop mechanisms. Unlike expensive commercial systems with AI navigation or IoT connectivity, this prototype focuses on core assistive functions—voice command mobility, safety, and reliability—while maintaining low cost and modular scalability. It serves as a practical foundation for future development in intelligent, voice-assisted mobility systems.

2.3.Research Gap

Most existing smart wheelchair systems focus on high-end automation features such as AI-based navigation, gesture or brain–computer control, and IoT connectivity, which significantly increase design complexity and cost. These advanced systems, while effective, are often not affordable or practical for small-scale implementation or educational purposes. There is a clear gap in developing low-cost, voice-operated wheelchair prototypes that combine essential automation, user-friendliness, and real-time safety features.

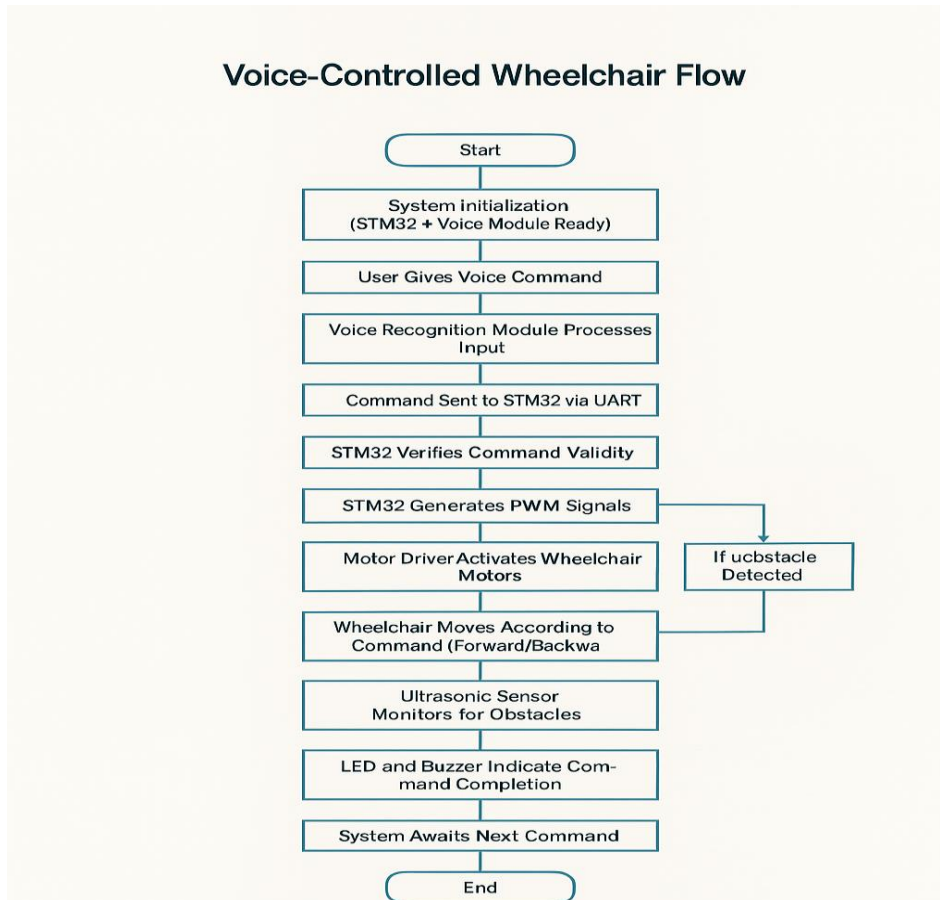
Additionally, many commercial models lack seamless integration of accurate voice recognition, obstacle detection, and emergency safety mechanisms within a compact and energy-efficient design. The proposed Voice-Controlled Wheelchair addresses these gaps by implementing a cost-effective, STM32-based system capable of real-time voice command processing, obstacle avoidance, and user feedback. This approach emphasizes affordability, reliability, and scalability, making it suitable for both academic research and real-world assistive mobility applications.

3.METHODOLOGY

3.1 Tools & Technologies Used

- Microcontroller: STM32F401RE — acts as the central processing unit, handling voice command input, motor control, and sensor feedback with real-time performance and low power consumption.
- Voice Recognition Module: Elechouse VR3 / EasyVR / VC-02 — processes the user's voice commands and sends corresponding digital signals to the STM32 via UART communication.
- Motor Driver Circuit: BTS7960 / L298N — used to drive the DC motors responsible for wheelchair movement, controlled through PWM signals from the STM32.
- Sensors: Ultrasonic sensor (HC-SR04) for obstacle detection to ensure safety during navigation, and emergency stop switch for immediate shutdown during hazards.
- Software Tools: STM32CubeIDE and Keil uVision for embedded programming and debugging; Proteus for simulation and circuit testing.
- Hardware Components: STM32F401RE microcontroller, voice recognition module, two DC motors for differential drive, motor driver circuits, ultrasonic sensors, emergency stop button, LED indicators, buzzer for feedback, and a 12V/24V rechargeable battery for power supply.

3.2 System Design



3.3 Architecture

- STM32F401RE Microcontroller: Serves as the central control unit, managing voice command processing, motor actuation, obstacle detection, and safety mechanisms in real time.
- Voice Recognition Module: Captures and interprets the user's spoken commands (e.g., Forward, Backward, Left, Right, Stop) and transmits them digitally to the STM32 via UART communication.
- Motor Drivers and DC Motors: Utilize BTS7960 or L298N motor driver circuits to control the differential drive motors that enable smooth and directional wheelchair movement.
- Ultrasonic Sensors and Emergency Stop System: Continuously monitor for obstacles and allow immediate halting of motion to prevent collisions or unsafe operation.
- Feedback Components (LED and Buzzer): Provide visual and audible confirmation of received commands and system status to enhance user awareness and reliability.

3.4. Software and Tools:

- Firmware Development: STM32CubeIDE and Keil uVision are used for programming the STM32F401RE microcontroller in Embedded C, including PWM generation, UART communication, and sensor interfacing.
- Simulation and Testing: Proteus Design Suite is utilized for simulating circuit behavior, verifying motor driver responses, and testing voice command logic before hardware implementation.
- Version Control: Git is used for managing firmware revisions, ensuring efficient code tracking, updates, and team collaboration.
- Debugging Tools: Integrated debugging features of STM32CubeIDE and serial monitoring tools are used to analyze real-time UART data between the microcontroller and voice module.
- Additional Software: Voice recognition libraries, ultrasonic sensor control modules, and STM32 HAL drivers are integrated to support accurate command processing and obstacle detection.

Hardware:

- Controller: STM32F401RE microcontroller serves as the central unit for processing voice commands, generating PWM signals, and managing motor and sensor operations.
- Voice Recognition Module: Elechouse VR3 / EasyVR / VC-02 module captures and interprets spoken commands, transmitting them to the STM32 via UART communication.
- Motor System: DC motors controlled through BTS7960 or L298N motor drivers enable smooth forward, backward, left, and right movement using differential drive control.
- Sensors: Ultrasonic sensors (HC-SR04) detect obstacles, while an emergency stop switch ensures immediate shutdown during unsafe conditions.
- Feedback Components: LED indicators and a buzzer provide real-time feedback for command acknowledgment and alert signals.
- Power Supply: A 12V/24V rechargeable battery with a regulated 5V output ensures stable power for the controller, sensors, and voice module during continuous operation.

- Laptop: Used for programming, firmware uploading via STM32CubeIDE, circuit simulation in Proteus, and real-time debugging during testing and prototyping.

3.5.Workflow

Start → System turns on and initializes → User gives a voice command → Voice module recognizes and sends it to STM32 → STM32 controls the motors through the driver → Wheelchair moves in the commanded direction → Ultrasonic sensor checks for obstacles → If obstacle found, wheelchair stops and buzzer alerts → If clear, it keeps moving → User says “Stop” or timeout occurs → Motors stop and system waits for the next command.

4.IMPLEMENTATION

4.1. Modules Description

- Voice Command Module: Captures and processes user speech commands such as Forward, Backward, Left, Right, and Stop for movement control.
- Motor Control Module: Uses the STM32 microcontroller and motor drivers to control the DC motors and enable smooth directional movement.
- Sensor and Safety Module: Includes ultrasonic sensors for obstacle detection and an emergency stop switch to prevent collisions.
- Feedback Module: Provides real-time responses using LEDs and a buzzer to confirm received commands and alert during obstacles.
- Power Management Module: Supplies stable power through a rechargeable battery system, ensuring continuous and reliable operation.

4.2. Algorithms/Processes

- Voice Command Detection: The microphone captures the user’s voice, and the recognition module converts it into a digital command.
- Command Processing: The STM32 microcontroller receives the command, verifies it, and decides the direction of movement.
- Motor Control: Based on the command, the motor driver activates the DC motors using PWM signals to move forward, backward, left, or right.
- Obstacle Detection: Ultrasonic sensors continuously check for obstacles; if detected, the system stops immediately and triggers the buzzer.

- Emergency Stop: When the user says “Stop” or presses the emergency switch, all motor operations halt for safety.

4.3.Source Code:

```
#include "main.h"

#include "usart.h"

#include "gpio.h"

#include "tim.h"

#include <string.h>

#include <stdbool.h>

/* Private defines - pin mapping (update if you change wiring) */

#define UART_HANDLE    huart1

// Motor A (Left)

#define DIR_A1_GPIO    GPIOA

#define DIR_A1_PIN     GPIO_PIN_2

#define DIR_A2_GPIO    GPIOA

#define DIR_A2_PIN     GPIO_PIN_3

#define EN_A_TIM_CHANNEL TIM_CHANNEL_1

#define EN_A_TIM_INSTANCE &htim2

// Motor B (Right) — Motor C is wired in parallel to Motor B outputs

#define DIR_B1_GPIO    GPIOB

#define DIR_B1_PIN     GPIO_PIN_4

#define DIR_B2_GPIO    GPIOB

#define DIR_B2_PIN     GPIO_PIN_6

#define EN_B_TIM_CHANNEL TIM_CHANNEL_2

#define EN_B_TIM_INSTANCE &htim2

// Ultrasonic HC-SR04
```

```

#define US_TRIG_GPIO    GPIOA

#define US_TRIG_PIN     GPIO_PIN_8

#define US_ECHO_GPIO    GPIOB

#define US_ECHO_PIN     GPIO_PIN_10


// Buzzer

#define BUZZER_GPIO     GPIOB

#define BUZZER_PIN      GPIO_PIN_9


// Emergency Stop (active LOW)

#define EMG_GPIO        GPIOC

#define EMG_PIN         GPIO_PIN_13


/* External handles (from CubeMX generated files) */

extern UART_HandleTypeDef huart1;

extern TIM_HandleTypeDef htim2;


/* Private variables */

volatile uint8_t rx_byte = 0;

volatile uint8_t cmd_ready = 0;

volatile bool avoid_left_next = true;


const uint32_t pwm_max = 1000; // matches TIM ARR = 999

uint32_t default_duty = 600; // start at 60% to limit current (tune later)


/* Function prototypes */

void SystemClock_Config(void);

void Error_Handler(void);

static void ensure_dwt_enabled(void);

static uint32_t micros(void);

static void ultrasonic_trigger(void);

```

```

static uint32_t ultrasonic_read_cm(void);

static void buzzer_on(void);

static void buzzer_off(void);

static void motors_stop(void);

static void set_motor_pwm(uint32_t duty);

static void motors_forward(void);

static void motors_backward(void);

static void motors_turn_left(void);

static void motors_turn_right(void);


/* Enable DWT cycle counter for micros() */
static void ensure_dwt_enabled(void)
{
    if (!(CoreDebug->DEMCR & CoreDebug_DEMCR_TRCENA_Msk)) CoreDebug->DEMCR |=
CoreDebug_DEMCR_TRCENA_Msk;

    if (!(DWT->CTRL & DWT_CTRL_CYCCNTENA_Msk)) DWT->CTRL |= DWT_CTRL_CYCCNTENA_Msk;
}


/* micros() using DWT->CYCCNT */
static uint32_t micros(void)
{
    uint32_t cycles = DWT->CYCCNT;

    uint32_t mhz = HAL_RCC_GetHCLKFreq() / 1000000U;

    if (mhz == 0) return 0;

    return cycles / mhz;
}


/* Ultrasonic trigger pulse */
static void ultrasonic_trigger(void)
{
    HAL_GPIO_WritePin(US_TRIG_GPIO, US_TRIG_PIN, GPIO_PIN_RESET);

    HAL_Delay(2);
}

```

```

    HAL_GPIO_WritePin(US_TRIG_GPIO, US_TRIG_PIN, GPIO_PIN_SET);

    uint32_t t = micros();

    while (micros() - t < 10) {} // 10 us pulse

    HAL_GPIO_WritePin(US_TRIG_GPIO, US_TRIG_PIN, GPIO_PIN_RESET);
}

/* Read ultrasonic distance in cm (polling) */
static uint32_t ultrasonic_read_cm(void)
{
    ultrasonic_trigger();

    uint32_t t0 = micros();
    while (HAL_GPIO_ReadPin(US_ECHO_GPIO, US_ECHO_PIN) == GPIO_PIN_RESET) {
        if (micros() - t0 > 30000U) return 999; // timeout
    }
    uint32_t t_rise = micros();
    while (HAL_GPIO_ReadPin(US_ECHO_GPIO, US_ECHO_PIN) == GPIO_PIN_SET) {
        if (micros() - t_rise > 30000U) break; // timeout
    }
    uint32_t t_fall = micros();

    uint32_t pulse = t_fall - t_rise; // microseconds
    // distance (cm) = (pulse_us * 0.0343) / 2
    float dist = (pulse * 0.0343f) / 2.0f;
    return (uint32_t)(dist + 0.5f);
}

/* Buzzer control */
static void buzzer_on(void) { HAL_GPIO_WritePin(BUZZER_GPIO, BUZZER_PIN, GPIO_PIN_SET); }
static void buzzer_off(void) { HAL_GPIO_WritePin(BUZZER_GPIO, BUZZER_PIN, GPIO_PIN_RESET); }

```

```
/* Motor helpers - two channels only (Motor C is hardware-paralleled to Motor B) */
```

```
static void motors_stop(void)
```

```
{  
    __HAL_TIM_SET_COMPARE(EN_A_TIM_INSTANCE, EN_A_TIM_CHANNEL, 0);  
    __HAL_TIM_SET_COMPARE(EN_B_TIM_INSTANCE, EN_B_TIM_CHANNEL, 0);  
}
```

```
static void set_motor_pwm(uint32_t duty)
```

```
{  
    if (duty > pwm_max) duty = pwm_max;  
    __HAL_TIM_SET_COMPARE(EN_A_TIM_INSTANCE, EN_A_TIM_CHANNEL, duty);  
    __HAL_TIM_SET_COMPARE(EN_B_TIM_INSTANCE, EN_B_TIM_CHANNEL, duty);  
}
```

```
static void motors_forward(void)
```

```
{  
    HAL_GPIO_WritePin(DIR_A1_GPIO, DIR_A1_PIN, GPIO_PIN_SET);  
    HAL_GPIO_WritePin(DIR_A2_GPIO, DIR_A2_PIN, GPIO_PIN_RESET);  
  
    HAL_GPIO_WritePin(DIR_B1_GPIO, DIR_B1_PIN, GPIO_PIN_SET);  
    HAL_GPIO_WritePin(DIR_B2_GPIO, DIR_B2_PIN, GPIO_PIN_RESET);  
  
    set_motor_pwm(default_duty);  
}
```

```
static void motors_backward(void)
```

```
{  
    HAL_GPIO_WritePin(DIR_A1_GPIO, DIR_A1_PIN, GPIO_PIN_RESET);  
    HAL_GPIO_WritePin(DIR_A2_GPIO, DIR_A2_PIN, GPIO_PIN_SET);  
  
    HAL_GPIO_WritePin(DIR_B1_GPIO, DIR_B1_PIN, GPIO_PIN_RESET);
```

```

HAL_GPIO_WritePin(DIR_B2_GPIO, DIR_B2_PIN, GPIO_PIN_SET);

set_motor_pwm(default_duty);
}

static void motors_turn_left(void)
{
    // left motor reverse, right motor forward
    HAL_GPIO_WritePin(DIR_A1_GPIO, DIR_A1_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(DIR_A2_GPIO, DIR_A2_PIN, GPIO_PIN_SET);

    HAL_GPIO_WritePin(DIR_B1_GPIO, DIR_B1_PIN, GPIO_PIN_SET);
    HAL_GPIO_WritePin(DIR_B2_GPIO, DIR_B2_PIN, GPIO_PIN_RESET);

    set_motor_pwm(default_duty);
}

static void motors_turn_right(void)
{
    // right motor reverse, left motor forward
    HAL_GPIO_WritePin(DIR_A1_GPIO, DIR_A1_PIN, GPIO_PIN_SET);
    HAL_GPIO_WritePin(DIR_A2_GPIO, DIR_A2_PIN, GPIO_PIN_RESET);

    HAL_GPIO_WritePin(DIR_B1_GPIO, DIR_B1_PIN, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(DIR_B2_GPIO, DIR_B2_PIN, GPIO_PIN_SET);

    set_motor_pwm(default_duty);
}

/* UART Rx complete callback (HAL) */
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)

```

```

{
    if (huart == &huart1) {
        cmd_ready = 1;
    }
}

/* Main program */
int main(void)
{
    /* MCU Init */
    HAL_Init();
    SystemClock_Config();

    /* Peripheral init - these functions are from CubeMX generated code */
    MX_GPIO_Init();
    MX_USART1_UART_Init();
    MX_TIM2_Init();

    /* Enable DWT for micros() */
    ensure_dwt_enabled();

    /* Start PWM channels (compare value 0 initially) */
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
    HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_2);

    /* Safety: start stopped */
    motors_stop();

    /* Start UART receive interrupt for single-byte commands */
    HAL_UART_Receive_IT(&huart1, (uint8_t *)&rx_byte, 1);

```

```

const uint32_t ULTRA_PERIOD_MS = 150; // measurement period

const uint32_t OBSTACLE_THRESHOLD_CM = 30;

uint32_t last_ultra_ts = HAL_GetTick();

while (1)
{
    /* Emergency Stop (active LOW on PC13). If pressed, kill motors and skip commands. */
    if (HAL_GPIO_ReadPin(EMG_GPIO, EMG_PIN) == GPIO_PIN_RESET) {
        motors_stop();
        buzzer_off();
        HAL_Delay(50);
        continue;
    }

    /* Command processing from HC-05 (single char commands) */
    if (cmd_ready) {
        cmd_ready = 0;
        uint8_t c = rx_byte;

        /* restart UART interrupt for next byte */
        HAL_UART_Receive_IT(&huart1, (uint8_t *)&rx_byte, 1);

        switch (c) {
            case 'F': motors_forward(); break;
            case 'B': motors_backward(); break;
            case 'L': motors_turn_left(); break;
            case 'R': motors_turn_right(); break;
            case 'S':
                default: motors_stop(); break;
        }
    }
}

```



```

/* Periodic ultrasonic check */
if ((HAL_GetTick() - last_ultra_ts) >= ULTRA_PERIOD_MS) {
    last_ultra_ts = HAL_GetTick();
    uint32_t dist = ultrasonic_read_cm();
    if (dist <= OBSTACLE_THRESHOLD_CM) {
        /* Obstacle detected: beep and avoidance maneuver */
        buzzer_on();
        HAL_Delay(120);
        buzzer_off();

        if (avoid_left_next) motors_turn_left();
        else motors_turn_right();
        avoid_left_next = !avoid_left_next;

        HAL_Delay(400); // turning duration - tune on your platform
        motors_stop();
    }
}

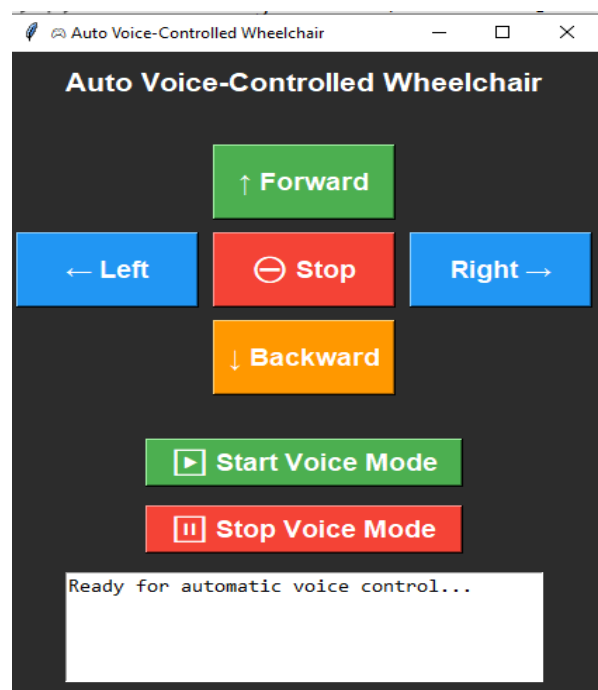
HAL_Delay(5); // small idle
}

/* Error_Handler implementation (optional - usually in main.c generated by Cube) */
void Error_Handler(void)
{
    __disable_irq();
    while (1) { }
}
...

```

5 . RESULTS AND DISCUSSION

5.1 control panel



6. CONCLUSION & FUTURE SCOPE

6.1 Conclusion

This project successfully demonstrates a functional Voice-Controlled Wheelchair prototype with key features such as real-time voice command processing, motorized movement control, obstacle detection, and emergency stop functionality. The system ensures smooth and safe operation through the integration of a speech recognition module, STM32 microcontroller, and ultrasonic sensors, providing users with hands-free mobility and enhanced independence. The design achieves its primary goal of developing a low-cost, reliable, and user-friendly assistive device suitable for individuals with physical disabilities.

Overall, the project establishes a strong foundation for future enhancements such as IoT integration for remote monitoring, AI-based voice recognition for improved accuracy, and autonomous navigation for fully intelligent mobility systems. This work highlights how embedded systems and automation can effectively improve accessibility, safety, and quality of life through innovative assistive technology.

6.2 Future Scope

- Integrate AI-based voice recognition for multiple languages and noisy environments.
- Add sensors for obstacle detection and safe navigation.
- Connect with smart home devices for enhanced user independence.
- Develop mobile app for monitoring, alerts, and remote control.
- Implement GPS navigation and emergency response features.
- Include health monitoring sensors for heart rate and posture.
- Optimize battery usage and enable solar-assisted charging.

REFERENCE

- STMicroelectronics. (2023). *RM0008 Reference Manual – STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx, STM32F107xx advanced ARM®-based 32-bit MCUs*. Retrieved from <https://www.st.com/en/microcontrollers-microprocessors.html>
- STMicroelectronics. (2024). *STM32Cube HAL and LL Drivers User Manual*. Retrieved from <https://www.st.com/en/embedded-software/stm32cubefw.html>
- STMicroelectronics. (2024). *UM2609: STM32CubeIDE User Guide*. Retrieved from https://www.st.com/resource/en/user_manual/dm00628133.pdf